# PRD: VaultAuth SSO (updated)

## 1. Header

| Field | Value |
|---|---|
| Project | VaultAuth SSO |
| ID / Track | PST_PD_154 / Product development |
| Batch / Level | 2026 / Level 4 (Stretch) |
| Duration | 4 weeks (3 working weeks + 1 buffer week) |
| Team | 2 students. **Backend + DevSecOps: Shaurya Jain (this document's owner).** Frontend: Apoorva Bhardwaj. Mentor: Mahati Ma'am |
| PRD version | **v2.0**, updated from v1.0 |
| Status | Decisions D1 to D14 approved. Items tagged **[Mentor review]** deviate from v1.0 and need mentor approval |

**Companion documents:** `README.md`, `architecture.md`, `auth.md` (API contract and examples), `usersmapping.md`, `schema.md`, `tech.md`, `guide.md`.

**How to read this document.** Decisions are in decision-record form (section 21). Anything not yet confirmed is under *Proposed (pending confirmation)* or *Open decisions*, never mixed with the decided items. Third-party facts I could not confirm from an official source are marked **verify**.

# 2. What changed from v1.0 and why

v1.0 was a client-only demo (localStorage, SHA-256 in the browser). That design cannot deliver its own success criteria: anyone can edit localStorage and become admin, a session cannot be revoked from another browser, and an unsalted fast hash is not a safe way to store passwords. v2.0 keeps the PRD's flows, screens, roles, and API shape, and moves the truth to a real backend.

| # | v1.0 | v2.0 | Why | Tag |
|---|---|---|---|---|
| 1 | SHA-256 in browser (`SubtleCrypto`) | **argon2id on the server**; browser sends the password over TLS only | SHA-256 is fast and unsalted by default, so it is cheap to brute-force | [Mentor review] |
| 2 | localStorage + `sample-data.json` | **Postgres** (Supabase primary, Neon fallback) | Roles, locks, and revocation are only real if the server enforces them | [Mentor review] |
| 3 | Mock OTP shown in a toast | **Real OTP by email** (Resend); optional TOTP upgrade | A code visible in the UI is not a second factor | [Mentor review] |

| | | | | |
|---|---|---|---|---|
| 4 | "Invalid credentials. Attempt N/3." and "Account locked. Contact administrator." | **One generic error** for every login failure | The old messages reveal which emails exist and how many tries remain | [Mentor review] |
| 5 | `ERR_DUPLICATE_EMAIL` on register | Generic "check your email" response | Same reason as row 4 | [Mentor review] |
| 6 | No email ownership check | Account is `PENDING` until the emailed code is verified | Otherwise anyone can make us email OTPs to a victim and burn the daily email cap | [Mentor review] |
| 7 | Anyone can register as admin | Public registration creates `user` only | Self-service admin is a privilege escalation | [Mentor review] |
| 8 | Lock rule contradicts itself (5 min vs. "contact administrator") | Auto-unlock after 5 min **and** admin unlock | Both PRD sentences are honoured | |
| 9 | Reset-state button for both roles | **CLI only**, run by the backend student (D15). No HTTP endpoint | A remote "wipe everything" endpoint on a real backend is a vulnerability | [Mentor review] |

| | | | | |
|---|---|---|---|---|
| 10 | Account lock and session revocation were "Good to have" | **Promoted to Must** | The success metric and CI suite test them | |
| 11 | Stack says "JavaScript/ Python" | **Python (FastAPI)** backend, vanilla JS + Bootstrap frontend | One backend language | |

# 3. Overview

VaultAuth SSO is an email + password authentication portal for **End Users** (employees) and **Enterprise Admins**. A valid password leads to a second step (a 6-digit code emailed to the user, or a TOTP code if the user enrolled an authenticator app). Only then does the server issue a session token. Admins can list and revoke any session, unlock accounts, and read the security audit log. A revoked session is rejected on its very next request.

- **Industry:** Cybersecurity. **Theme:** Identity & Access.
- **One-line pitch:** "For enterprise teams seeking secure sign-on without complex OAuth infrastructure, VaultAuth SSO provides argon2id-hashed credentials, email-verified 2FA, and active session revocation that takes effect on the next request."
- **Optional (cut first):** 2-screen React Native (Expo Go) Corporate Digital ID Card that lists sessions and revokes one. WebAuthn/ FIDO2 stays a reading-only concept card.

# 4. Problem statement

Organizations store credentials weakly, skip multi-factor checks, and cannot kill a compromised session quickly.

- **Concrete pain:** password leaks, session hijacking, and no way for an admin to revoke a session instantly.
- **Solved when:**
    1. Passwords are stored only as argon2id hashes, and the plaintext is never stored or logged.
    2. A correct email + password triggers a real second factor before any session exists.
    3. Active sessions appear in an admin console, and a revoked session is rejected on its next request (end-to-end under 30 s).

# 5. Users and roles

| Capability | End User | Enterprise Admin |
|---|---|---|
| Register (creates a `user`) and verify email | Yes | Yes (admins are seeded or promoted, see D7) |
| Log in with email + 2FA | Yes | Yes |
| Enroll TOTP (optional) | Yes | Yes |
| View own sessions and Digital ID Card | Yes | Yes |
| Revoke **own** session / log out | Yes | Yes |
| Revoke **any** user's session | No | Yes |
| View global security audit log | No | Yes |

| | | |
|---|---|---|
| Unlock, enable/disable an account; promote a user to admin | No | Yes |
| Reset demo state | **CLI only**, run by the backend student (D15). Never over HTTP | **CLI only** (D15) |

This resolves a v1.0 contradiction: the role table said users cannot revoke, while BR-04 said they can revoke their own. Users revoke only their own.

# 6. Scope and cut list

**Must have (graded, never cut):** registration with email verification; argon2id; login with generic errors; email OTP 2FA; sessions with idle and absolute expiry; logout; own-session list; account lock; session revocation (own and admin); admin audit log; Postgres persistence; CI test suite; deployed backend and README; responsive Bootstrap UI (teammate).

**Cut list, first to cut → last to cut:**

| Order | Item | Why this order |
|---|---|---|
| 1 | Expo Corporate Digital ID Card | Lowest value for the auth goals |
| 2 | TOTP enrollment and recovery codes | Email OTP already satisfies the PRD |
| 3 | Neon fallback sync job | Small CI job, kept longest. If it is cut there is **no backup at all** (Supabase free has none, **verify**), so cut it last |

**Out of scope:** SMS gateway, Okta/Auth0 deployment, YubiKey hardware parsing, real WebAuthn.

# 7. Functional requirements

| ID | Requirement | Priority | Was | Acceptance test |
|---|---|---|---|---|
| FR-01 | **Registration.** Email + password. Server hashes with argon2id. Account is `PENDING`. Response is always the same ("check your email") | Must | FR-01 | New address: row has `$argon2id$...` hash, status `PENDING`. Existing address: identical response body and status code, plus an "already registered" email |
| FR-02 | **Email verification.** 6-digit code, hashed at rest, 10 min expiry, 3 wrong tries invalidate it. Resend capped at 3, then cooldown | Must | new | Correct code → `ACTIVE`. 4th resend → `429 ERR_RATE_LIMITED` |

| | | | | |
|---|---|---|---|---|
| FR-03 | **Login.** Verifies email + password. Every failure returns the same 401 body | Must | FR-02 | Unknown email, wrong password, locked account, and disabled account all return identical status and body. A correct password on a `PENDING` account returns `VERIFY_EMAIL_REQUIRED` instead (P12) |
| FR-04 | **2FA.** After a correct password the server issues a challenge (email OTP, or TOTP if enrolled). A correct code creates the session. 3 wrong codes invalidate the challenge and force re-login | Must | FR-03 | Correct code → Bearer token. 3 wrong → challenge dead, `ERR_INVALID_OTP`, must log in again |
| FR-05 | **Sessions.** Opaque 256-bit token, only its hash stored. 30 min idle and 8 h absolute expiry. Logout revokes | Must | FR-06 | Expired token → 401. Logout → token unusable |

| | | | | |
|---|---|---|---|---|
| FR-06 | **Account lock.** 3 consecutive wrong passwords → `LOCKED` for 5 min, then auto-unlock. Admin can unlock | Must (was Good) | FR-04 | 3 failures → login fails even with the right password. After 5 min it works. 20 parallel failures count exactly 3 |
| FR-07 | **Session revocation.** Users revoke own sessions. Admins revoke any | Must (was Good) | FR-05 | Revoke → the very next request with that token returns 401 |
| FR-08 | **Admin audit log.** Security events, newest first, filterable by event | Must | role table | Non-admin gets `403`. Admin sees login failures, locks, revocations |
| FR-09 | **Admin user actions.** Unlock, enable/ disable, promote. Disabling revokes that user's live sessions | Should | new | Each action is audited; non-admin gets `403`; disabled user's token fails on its next request |

| | | | | |
|---|---|---|---|---|
| FR-10 | **Persistence.** Postgres replaces localStorage and `sample-data.json` | Must | FR-06 | Restart the API → accounts, logs, and sessions persist |
| FR-11 | **TOTP.** Optional enrollment (QR), replaces email OTP at login, recovery codes | Could (cut #2) | new | Enrolled user is asked for a TOTP code, not an email code |
| FR-12 | **Fallback DB sync.** Scheduled dump/restore from CI to Neon | Should (cut #3) | new | Fallback row counts match primary after a run |
| FR-13 | **Expo ID card.** Two screens: session list, revoke | Could (cut #1) | stretch | Revoke from the phone kills a web session |

# 8. Architecture

## 8.1 System at a glance

```
flowchart LR
  subgraph Clients
```

```
    WEB["Web frontend: static on Vercel, CSP and security
    EXPO["Expo ID card (stretch, cut first)"]
  end
  subgraph Render["Render free web service (Docker image,
    API["FastAPI: auth, 2FA, sessions, roles, audit"]
  end
  subgraph DB["Postgres, private schema vaultauth"]
    PRIMARY[("Supabase primary")]
    FALLBACK[("Neon fallback")]
  end
  MAIL["Resend, called inside the request"]
  CI["GitHub Actions: tests, deploy, keepalive, sync, pur
  WEB -->|"HTTPS + Bearer token"| API
  EXPO -.->|"HTTPS + Bearer token"| API
  API -->|"vault_app via transaction pooler"| PRIMARY
  API -->|"OTP and notice emails"| MAIL
  CI -->|"ping /health"| API
  CI -->|"alembic as vault_migrator"| PRIMARY
  CI -->|"pg_dump as vault_backup"| PRIMARY
  CI -->|"restore as vault_migrator"| FALLBACK
  CI -->|"purge as vault_app"| PRIMARY
```

Component roles:

| Component | Role | Where it runs |
|---|---|---|
| Web frontend | Static pages (HTML/CSS/ vanilla JS/Bootstrap 5), calls the API with a Bearer token | Vercel (teammate), served with a CSP and security headers (D20) |
| FastAPI backend | All security decisions: hashing, 2FA, sessions, roles, audit | Render free web service, Docker image also provided |
| Postgres | Single source of truth for identities, sessions, audit. Private schema `vaultauth`, three database roles (D18, P25) | Supabase (primary), Neon (fallback) |

| | | |
|---|---|---|
| Resend | Sends OTP and verification emails through an `EmailSender` interface, called **inside the request** (D19) | Resend free tier, on your verified domain |
| GitHub Actions | Tests, deploy, and cron jobs: keepalive ping, fallback sync, purge | GitHub |

## 8.2 Login and session flow

```
sequenceDiagram
  autonumber
  participant C as Client
  participant A as FastAPI
  participant D as Postgres
  participant M as Resend
  C->>A: POST /auth/login (email, password)
  A->>D: load user, check status, locked_until
  alt unknown email, wrong password, locked, or disabled
    A-->>C: 401 ERR_INVALID_CREDENTIALS (same body always
    A->>D: audit event, atomic failed_attempts update
  else password ok but account PENDING
    A-->>C: 200 VERIFY_EMAIL_REQUIRED (resend caps apply)
  else password ok and account ACTIVE
    A->>D: invalidate old challenge, insert new (hashed c
    A->>M: email 6-digit code (or ask for TOTP)
    A-->>C: 200 challenge_id, OTP_REQUIRED
    C->>A: POST /auth/2fa/verify (challenge_id, code)
    A->>D: attempts+1, compare hash, consume challenge
    A->>D: insert session (token hash, idle and absolute
    A-->>C: 200 opaque Bearer token (shown once)
  end
  C->>A: any request with Authorization header
  A->>D: hash token, require ACTIVE and not expired, bump
```

**Terms used consistently in this document.** A *code* is the 6-digit value

the user types (emailed OTP or TOTP). A *challenge* is the server-side record that waits for that code (`otp_challenges`). A *token* is the opaque Bearer value the server returns after a correct code. A *session* is the database row that the token's hash points to. There are no idempotency keys in this system.

## 8.3 Account and session lifecycles

```
stateDiagram-v2
  [*] --> PENDING: register
  PENDING --> ACTIVE: email code verified
  ACTIVE --> LOCKED: 3 consecutive failures
  LOCKED --> ACTIVE: 5 min elapsed or admin unlock
  ACTIVE --> DISABLED: admin disables
  DISABLED --> ACTIVE: admin enables
  state "Session" as S {
    [*] --> SACTIVE
    SACTIVE --> SREVOKED: user or admin revokes
    SACTIVE --> SEXPIRED: idle 30 min or absolute 8 h
  }
```

## 8.4 Runtime, data access, and trust boundaries

Two kinds of "role" exist and are never mixed up in these documents: a **user role** (`user` or `admin`, a column in `users`) and a **database role** (`vault_app`, `vault_migrator`, `vault_backup`, a Postgres login).

```
flowchart TB
  subgraph who["Who connects, and as which database role"
    APP["FastAPI app and CLI: purge, promote-admin, revok
    MIG["Alembic migrations and CLI reset"]
    BK["CI fallback sync, dump side"]
    ANON["Supabase Data API roles: anon, authenticated"]
  end
  subgraph S["Private schema vaultauth, owned by vault_mi
    T[("users, sessions, otp_challenges, audit_logs, ..."
  end
```

```
APP -->|"vault_app: SELECT INSERT UPDATE DELETE only"|
MIG -->|"vault_migrator: owner, DDL, TRUNCATE"| T
BK -->|"vault_backup: SELECT only"| T
ANON -.->|"no USAGE on the schema: blocked"| T
```

| Topic | Rule | Status |
|---|---|---|
| Database roles | `vault_app` can only SELECT, INSERT, UPDATE, DELETE. `vault_migrator` owns the schema, runs migrations, and runs the CLI `reset` (it alone can TRUNCATE). `vault_backup` can only SELECT. A leaked app credential cannot drop, alter, or truncate anything | D18, tested |
| Private schema | Our tables live in `vaultauth`, not `public`, so Supabase's auto-generated Data API (which exposes `public` by default) cannot reach them. The Data API is also switched off, because we never use it. Tested: a Data-API-style `anon` role cannot read the users table | P25 |
| Startup self-check | In production the app refuses to start if its database role could create objects, truncate, or has no DML rights. Tested against all three roles and `postgres` | P29 |
| Pooler mode | The app uses Supabase's **transaction-mode** pooler. That mode does not support prepared statements, so the driver must run with `prepare_threshold=None`. Migrations and dumps use a direct or session connection | P26, verified fact |
| Process model | One uvicorn worker. argon2 runs in a threadpool behind a semaphore of `HASH_CONCURRENCY`, so a login burst cannot freeze `/health` or the keepalive | P22 |

| | | |
|---|---|---|
| Email | Sent inside the request. Failure returns `503`. A background task could silently lose codes when Render restarts, and Render free has no second process for an outbox | D19 |
| Regions | Every authenticated request is a database round trip with a write, so Render and Supabase should be in the same or nearest region | P23, **verify** regions |
| Free capacity | The keepalive keeps one Render service awake for about 744 of the 750 free hours, so no second always-on Render service fits | P24, **verify** |
| Browser protections | Frontend on Vercel with real response headers (CSP and others). GitHub Pages cannot set headers, and a meta-tag CSP is weaker | D20, P28 |

# 9. Tech stack

| Category | Choice | Notes |
|---|---|---|
| Backend and security | Python, FastAPI, `argon2-cffi`, `pyotp` (TOTP), Alembic, Pydantic | argon2id only. No bcrypt, no SHA-256 for passwords |
| Data and infra | Postgres (Supabase primary, Neon fallback) with three database roles, Docker, Render free, GitHub Actions | Database vendor coupling is just three connection URLs |
| Frontend and clients | HTML5, CSS3, vanilla JS, Bootstrap 5, hosted on Vercel with security headers (teammate). Expo Go (stretch) | Frontend is a pure API client |

# 10. Data model

All identities live in our own Postgres (no Supabase Auth), so one backup contains everything. The DDL below was executed against PostgreSQL 16 and tested (section 17.3).

```
erDiagram
  users ||--o{ otp_challenges : "owns"
  users ||--o{ sessions : "owns"
  users ||--o{ recovery_codes : "owns"
  users ||--o{ email_sends : "triggers"
  users ||--o{ audit_logs : "actor or target"
  otp_challenges ||--o| sessions : "same user only (compo
  users {
    uuid id PK
    text email UK
    text password_hash
    text role
    text status
    smallint failed_attempts
    timestamptz locked_until
    bytea totp_secret_enc
    boolean totp_enabled
  }
  otp_challenges {
    uuid id PK
    uuid user_id FK
    text purpose
    text code_hash
    smallint attempts
    timestamptz expires_at
  }
  sessions {
    uuid id PK
    uuid user_id FK
    uuid challenge_id FK
    char token_hash UK
    text status
    timestamptz absolute_expires_at
    uuid revoked_by FK
    text revoke_reason
```

```
  }
  audit_logs {
    bigint id PK
    text event
    uuid actor_user_id FK
    uuid target_user_id FK
    jsonb details
  }
  rate_limits {
    text bucket PK
    timestamptz window_start PK
    int hits
  }
```

```
-- VaultAuth SSO schema v1 (PostgreSQL 15+). gen_random_u
CREATE TABLE users (
  id                uuid PRIMARY KEY DEFAULT gen_random_uu
  email             text NOT NULL,
  password_hash     text NOT NULL,
  role              text NOT NULL DEFAULT 'user',
  status            text NOT NULL DEFAULT 'PENDING',
  failed_attempts   smallint NOT NULL DEFAULT 0,
  locked_until      timestamptz,
  totp_secret_enc   bytea,
  totp_enabled      boolean NOT NULL DEFAULT false,
  created_at        timestamptz NOT NULL DEFAULT now(),
  updated_at        timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT users_email_norm   CHECK (email = lower(btri
  CONSTRAINT users_hash_argon2  CHECK (password_hash LIKE
  CONSTRAINT users_role         CHECK (role IN ('user','a
  CONSTRAINT users_status       CHECK (status IN ('PENDIN
  CONSTRAINT users_failed_range CHECK (failed_attempts BE
  CONSTRAINT users_lock_has_ts  CHECK (status <> 'LOCKED'
  CONSTRAINT users_totp_has_key CHECK (NOT totp_enabled O
);
CREATE UNIQUE INDEX users_email_uq ON users (email);

CREATE TABLE otp_challenges (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid
```

```
  user_id        uuid NOT NULL REFERENCES users(id) ON DE
  purpose        text NOT NULL,
  code_hash      text NOT NULL,
  attempts       smallint NOT NULL DEFAULT 0,
  expires_at     timestamptz NOT NULL,
  consumed_at    timestamptz,
  invalidated_at timestamptz,
  created_at     timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT otp_purpose   CHECK (purpose IN ('EMAIL_VERI
  CONSTRAINT otp_attempts  CHECK (attempts BETWEEN 0 AND
  CONSTRAINT otp_exp_after CHECK (expires_at > created_at
  CONSTRAINT otp_one_end   CHECK (NOT (consumed_at IS NOT
  CONSTRAINT otp_user_id_uq UNIQUE (user_id, id)
);
CREATE INDEX otp_expires ON otp_challenges (expires_at);
-- at most one live challenge per user+purpose
CREATE UNIQUE INDEX otp_one_live ON otp_challenges (user_
  WHERE consumed_at IS NULL AND invalidated_at IS NULL;

CREATE TABLE sessions (
  id                  uuid PRIMARY KEY DEFAULT gen_random
  user_id             uuid NOT NULL REFERENCES users(id)
  challenge_id        uuid NOT NULL,
  token_hash          char(64) NOT NULL,
  device              text NOT NULL DEFAULT 'unknown',
  ip                  inet,
  status              text NOT NULL DEFAULT 'ACTIVE',
  created_at          timestamptz NOT NULL DEFAULT now(),
  last_seen_at        timestamptz NOT NULL DEFAULT now(),
  idle_expires_at     timestamptz NOT NULL,
  absolute_expires_at timestamptz NOT NULL,
  revoked_at          timestamptz,
  revoked_by          uuid REFERENCES users(id),       --
  revoke_reason       text,
  CONSTRAINT sessions_status   CHECK (status IN ('ACTIVE'
  CONSTRAINT sessions_tok_hex  CHECK (token_hash ~ '^[0-9
  CONSTRAINT sessions_revoked_ts     CHECK ((status = 'RE
  CONSTRAINT sessions_revoked_reason CHECK ((status = 'RE
  CONSTRAINT sessions_reason_vals    CHECK (revoke_reason
  CONSTRAINT sessions_reason_actor   CHECK (revoke_reason
```

```
  CONSTRAINT sessions_self_actor     CHECK (revoke_reason
  CONSTRAINT sessions_abs_gt   CHECK (absolute_expires_at
  CONSTRAINT sessions_idle_le  CHECK (idle_expires_at <=
  -- isolation: the challenge that produced a session mus
  CONSTRAINT sessions_challenge_same_user
    FOREIGN KEY (user_id, challenge_id) REFERENCES otp_ch
);
CREATE UNIQUE INDEX sessions_token_uq ON sessions (token_
CREATE INDEX sessions_user_status ON sessions (user_id, s
CREATE INDEX sessions_active_created ON sessions (created
CREATE INDEX sessions_abs_exp ON sessions (absolute_expir

CREATE TABLE recovery_codes (          -- TOTP (cut-list
  id        uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id   uuid NOT NULL REFERENCES users(id) ON DELETE
  code_hash text NOT NULL,
  used_at   timestamptz,
  UNIQUE (user_id, code_hash)
);

CREATE TABLE email_sends (             -- feeds per-accou
  id        bigserial PRIMARY KEY,
  user_id   uuid REFERENCES users(id) ON DELETE SET NULL,
  purpose   text NOT NULL,
  sent_at   timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT email_purpose CHECK (purpose IN ('EMAIL_VERI
);
CREATE INDEX email_sends_budget   ON email_sends (sent_at
CREATE INDEX email_sends_throttle ON email_sends (user_id

CREATE TABLE rate_limits (             -- fixed-window co
  bucket       text NOT NULL,          -- e.g. 'register:
  window_start timestamptz NOT NULL,
  hits         integer NOT NULL DEFAULT 1,
  PRIMARY KEY (bucket, window_start),
  CONSTRAINT rl_hits CHECK (hits > 0)
);
CREATE INDEX rate_limits_window ON rate_limits (window_st

CREATE TABLE audit_logs (
```

```
  id              bigserial PRIMARY KEY,
  ts              timestamptz NOT NULL DEFAULT now(),
  event           text NOT NULL,
  actor_user_id   uuid REFERENCES users(id) ON DELETE SET
  target_user_id uuid REFERENCES users(id) ON DELETE SET
  ip              inet,
  details         jsonb NOT NULL DEFAULT '{}',
  CONSTRAINT audit_event CHECK (event ~ '^[A-Z0-9_]{3,64}
);
CREATE INDEX audit_ts      ON audit_logs (ts DESC);
CREATE INDEX audit_event   ON audit_logs (event, ts DESC)
CREATE INDEX audit_target  ON audit_logs (target_user_id,
```

**Roles and schema.** The DDL runs inside the private schema `vaultauth` (each database role's `search_path` points there, so the SQL needs no prefix). `roles.sql` in `schema.md` creates the schema, the three roles, and default privileges so later migrations need no extra GRANTs. It runs once on the primary **and** on the fallback.

**Invariants enforced by the database**

| Invariant | Mechanism |
|---|---|
| Emails are lowercase, unique | `users_email_norm` CHECK + unique index |
| Passwords are argon2id only (no SHA-256 hex, no bcrypt can be stored) | `users_hash_argon2` CHECK |
| A `LOCKED` user always has `locked_until` | `users_lock_has_ts` CHECK |
| A session can only cite an OTP challenge **of the same user** | Composite FK `(user_id, challenge_id)` → `otp_challenges (user_id, id)` |
| At most one live challenge per user and purpose | Partial unique index `otp_one_live` |
| OTP tries never exceed 3 | `otp_attempts` CHECK |

| | |
|---|---|
| Revoked sessions always carry timestamp and reason; `SELF` means the owner, `SYSTEM` means no human | `sessions_*` CHECKs |
| Idle expiry never exceeds absolute expiry | `sessions_idle_le` CHECK |

**Not enforceable in SQL (enforced in code and covered by tests):** that an `ADMIN` revoker really has role `admin`; the 30-day purge; the email daily budget.

# 11. API and service contract

Base path `/api/v1`. All bodies are JSON. Errors use one envelope: `{"error": {"code": "...", "message": "...", "request_id": "..."}}`.

## 11.1 Endpoints and who may call them

| Endpoint | Access | Purpose |
|---|---|---|
| `POST /auth/register` | **Open**, rate-limited per IP | FR-01 |
| `POST /auth/verify-email` | **Open**, rate-limited | FR-02 |
| `POST /auth/verify-email/resend` | **Open**, capped 3 then cooldown | FR-02 |
| `POST /auth/login` | **Open**, rate-limited | FR-03 |
| `POST /auth/2fa/verify` | **Open**, rate-limited (needs a valid `challenge_id`) | FR-04 |

| | | |
|---|---|---|
| `GET /health` | **Open**, minimal body, checks the DB | Keepalive and monitoring |
| `/docs`, `/openapi.json` | **Disabled in production** (`DOCS_ENABLED=false`). The spec is shared with the frontend as a CI artifact | |
| `POST /auth/logout` | Authenticated | FR-05 |
| `GET /me`, `GET /me/sessions` | Authenticated | Own profile, own sessions |
| `DELETE /sessions/{id}` | Authenticated: own session. Admin: any | FR-07 |
| `POST /me/totp/enroll`, `/confirm`, `/disable` | Authenticated | FR-11 (cut #2) |
| `GET /admin/sessions`, `GET /admin/audit-logs` | Admin | FR-07, FR-08 |
| `POST /admin/users/{id}/unlock`, `/status`, `/role` | Admin | FR-09 |

## 11.2 Mapping from the v1.0 function contract

| v1.0 | v2.0 |
|---|---|
| `registerUser` | `POST /auth/register` (generic response; `ERR_WEAK_PASSWORD` is the only visible error) |

| | |
|---|---|
| `authenticateUser` → `OTP_REQUIRED` | `POST /auth/login` → `200 {challenge_id, state: "OTP_REQUIRED"}` |
| `verifyOTP` → session token | `POST /auth/2fa/verify` → `200 {token}` (shown once) |
| `revokeSession` | `DELETE /sessions/{id}` |

### 11.3 Error contract

| Code | HTTP | When |
|---|---|---|
| `ERR_INVALID_CREDENTIALS` | 401 | Any login failure: unknown email, wrong password, locked, disabled (or pending with a wrong password) |
| `ERR_WEAK_PASSWORD` | 422 | Fails the password policy |
| `ERR_VALIDATION` | 422 | Malformed input |
| `ERR_INVALID_OTP` | 401 | Wrong, expired, or already-invalidated code |
| `ERR_UNAUTHENTICATED` | 401 | Missing, expired, or revoked token |
| `ERR_FORBIDDEN` | 403 | Non-admin calls an admin endpoint |
| `ERR_NOT_FOUND` | 404 | Unknown resource, **and a non-admin touching someone else's session** (proposed, see 21.2) |
| `ERR_RATE_LIMITED` | 429 | IP or per-account limit, resend cap, cooldown |
| `ERR_EMAIL_UNAVAILABLE` | 503 | Email provider down or daily budget spent |

Removed from the client-visible contract: `ERR_ACCOUNT_LOCKED` and `ERR_DUPLICATE_EMAIL`. Both still appear in the **audit log** as events.

## 12. Business rules

| ID | Rule |
|---|---|
| BR-01 | Plaintext passwords are never stored, logged, or returned. Only argon2id hashes are stored. Logs never contain tokens, OTPs, or hashes |
| BR-02 | A correct password never creates a session. Only a verified second factor does |
| BR-03 | 3 consecutive wrong passwords set `LOCKED` for 5 min. The counter update is one atomic SQL statement. Correct login resets the counter |
| BR-04 | Users revoke only their own sessions. Admins revoke any |
| BR-05 | No client-visible message may reveal whether an email exists, is locked, or how many attempts remain |
| BR-06 | Session tokens are random 256-bit values. Only a SHA-256 of the token is stored (safe because the token is high-entropy; this is the only use of SHA-256) |
| BR-07 | Failover to the fallback DB revokes **all** sessions (reason `SYSTEM`) |
| BR-08 | Audit logs and expired or revoked sessions older than 30 days are purged |
| BR-09 | Public registration always creates role `user` |
| BR-11 | The application connects only as `vault_app` and refuses to start with a more powerful database role (P29) |
| BR-10 | Admin rights come only from the CLI (`promote-admin`) or from an existing admin. A lock blocks **new logins only**; it never kills a live session (otherwise an attacker could log a victim out by failing logins). `DISABLED` kills sessions at once |

# 13. Auth, roles, and data safety

### Responsibility split

| Concern | Backend (Shaurya) | Frontend (Apoorva) |
|---|---|---|
| Hashing, 2FA, sessions, roles, locks | All of it | None. The browser never hashes |
| Token handling | Issues, validates, revokes | Keeps the token in memory or `sessionStorage`, sends `Authorization: Bearer ...`, drops it on any 401 |
| Output safety | JSON only, strict CORS allowlist, security headers | Escape email, device, and IP before DOM insertion (XSS). Use `textContent`, not `innerHTML` |
| Browser protections | None (API only) | Vercel response headers: CSP with `script-src 'self'` and `connect-src` limited to the API origin, `frame-ancestors 'none'`, `Referrer-Policy`, `X-Content-Type-Options`, `Permissions-Policy`. Self-host Bootstrap or load it with SRI (P28) |
| Error display | Stable codes, generic messages | Show the message as given. Never add "attempt N/3" |

**Sessions.** Opaque Bearer tokens, validated against Postgres on every request. This is why revocation is immediate (JWTs would stay valid until expiry).

# 14. Error handling and edge cases

| Scenario | Behaviour |
|---|---|
| Wrong password, unknown email, locked, or disabled account | Same 401 `ERR_INVALID_CREDENTIALS`. An unknown email still costs one dummy argon2 verification so timing does not leak existence (proposed) |
| Correct password on a `PENDING` account | Sends a verification code (resend caps apply) and returns `VERIFY_EMAIL_REQUIRED`. The caller already proved the password (proposed) |
| Wrong OTP 3 times | Challenge invalidated. User must log in again |
| Email provider down, or daily budget used up | `503 ERR_EMAIL_UNAVAILABLE`. TOTP users can still log in |
| Resend cap reached | `429 ERR_RATE_LIMITED` with a retry time |
| Register or login feels slow | Email is sent inside the request (D19), so the provider's latency adds to hashing time. The UI shows a spinner |
| Server was asleep (Render free) | First request takes up to about a minute (**verify**). State is never kept in memory, so waking up loses nothing |
| Primary DB paused or down | `/health` fails. Operator follows the failover runbook (section 18.3) |
| Two parallel bad logins | Row lock serializes the counter. Counted once each, locks exactly at 3 |
| Browser storage full | Not applicable to security state; only the token lives in the browser |

# 15. Limits and abuse controls

| Control | Default (env-configurable, proposed values) |
|---|---|
| Register per IP | 5 per hour |
| Login per IP | 20 per minute |
| OTP tries per challenge | 3 |
| OTP and verification resends | 3, then 15 min cooldown |
| Email daily budget | 80 of Resend's 100/day (**verify**). Alert when 80% of that budget (64 emails) is used |
| Password length | 12 to 128 bytes, no composition rules |
| Concurrent argon2 hashes | 2 (Render free has 512 MB RAM, **verify**) |
| Request body | 16 KB |

Counters live in Postgres (`rate_limits`, `email_sends`), so they survive restarts and work with several server processes.

## 16. Configuration contract

Every setting comes from the environment. No secrets in the repo.

| Variable | Default | Purpose |
|---|---|---|
| `APP_ENV` | `production` | `development` or `production` |
| `DATABASE_URL` | none | App role `vault_app`, **transaction-mode pooled** URL, prepared statements off |

| | | |
|---|---|---|
| `DATABASE_MIGRATOR_URL` | none | `vault_migrator`, **direct or session** URL, for Alembic and the CLI `reset` (never the transaction pooler) |
| `DATABASE_BACKUP_URL` | none | `vault_backup`, **direct or session** URL, for `pg_dump` in CI |
| `FALLBACK_DATABASE_URL` | none | Neon `vault_migrator` URL, the target of the sync restore |
| `CORS_ALLOWED_ORIGINS` | none | Comma-separated frontend origins |
| `TRUSTED_PROXY_COUNT` | `1` | How many proxy hops to trust for the client IP |
| `DOCS_ENABLED` | `false` | Expose `/docs` and `/openapi.json` |
| `SESSION_IDLE_MINUTES` / `SESSION_ABSOLUTE_HOURS` | `30` / `8` | Session lifetime |
| `LOCK_THRESHOLD` / `LOCK_MINUTES` | `3` / `5` | Account lock |
| `OTP_TTL_MINUTES` / `OTP_MAX_ATTEMPTS` | `10` / `3` | OTP |
| `OTP_RESEND_MAX` / `OTP_RESEND_COOLDOWN_MINUTES` | `3` / `15` | Resend cap |
| `ARGON2_TIME_COST` / `ARGON2_MEMORY_KIB` / `ARGON2_PARALLELISM` | `2` / `19456` / `1` | OWASP minimum profile (**verify**) |

| | | |
|---|---|---|
| `WEB_CONCURRENCY` | `1` | Number of uvicorn workers (keep 1 on 512 MB) |
| `HASH_CONCURRENCY` | `2` | Max parallel hashes |
| `PASSWORD_MIN_LENGTH` / `PASSWORD_MAX_BYTES` | `12` / `128` | Password policy |
| `RATE_REGISTER_PER_HOUR_IP` / `RATE_LOGIN_PER_MIN_IP` | `5` / `20` | Abuse limits |
| `EMAIL_PROVIDER` / `EMAIL_FROM` / `RESEND_API_KEY` | `resend` / none / none | Email |
| `EMAIL_DAILY_BUDGET` | `80` | Self-imposed cap below the provider's |
| `TOTP_ENC_KEY` | none | Key that encrypts TOTP secrets at rest (only if FR-11 ships) |
| `MAX_BODY_BYTES` | `16384` | Request body limit |
| `RETENTION_DAYS` | `30` | Purge age |
| `LOG_LEVEL` | `INFO` | Structured JSON logs |

**CI secrets:** `HEALTH_URL`, `DATABASE_URL`, `DATABASE_MIGRATOR_URL`, `DATABASE_BACKUP_URL`, `FALLBACK_DATABASE_URL`. Each is scoped to the one workflow that needs it (P30).

# 17. Testing

## 17.1 Strategy

- **pytest against a real Postgres** (service container in CI). No mocked database.
- TOTP codes are computed in tests from a seeded secret, so no human is needed.
- The email sender is replaced by a recording fake in tests only.

## 17.2 Critical-path suite (success metric 2)

Register + verify; login generic errors (all four failure types identical); email OTP happy path; 3 wrong OTPs; lock at 3 with 20 parallel failures; auto-unlock; own vs. admin revoke; revoked token rejected on the next request; expired token rejected; non-admin blocked from admin endpoints; user A cannot see or revoke user B's sessions; rate limits; purge.

## 17.3 Already executed: database layer

I ran the DDL in section 10 on PostgreSQL 16. The schema suite has **37 checks, all passed**. Four more suites ran on top of it: **16 of 16** for the hot-path SQL statements (`schema.md` section 6), **6 of 6** for the retention purge, and a 3-step test of the fallback sync. Highlights:

| Area | Checked |
|---|---|
| Constraints | Duplicate and uppercase emails rejected; SHA-256 hex and bcrypt hashes rejected; bad role; `LOCKED` without `locked_until`; TOTP flag without secret; OTP tries above 3; idle expiry beyond absolute |
| Isolation | A session for user B citing user A's challenge → foreign-key violation |
| Challenges | Second live challenge for the same user and purpose rejected; allowed after invalidation |

| | |
|---|---|
| Revocation | Revoke without reason, `ADMIN` without actor, `SELF` by someone else, `SYSTEM` naming a human: all rejected. Valid admin revoke accepted. Revoked session fails the per-request validity query at once. "Revoke all" by `SYSTEM` works |
| Concurrency | 20 simultaneous wrong logins → counter exactly 3, `LOCKED`. 50 simultaneous rate-limit hits → exactly 50 |
| Retention | 30-day purge removes only old rows; user delete cascades |
| Indexes (at 2,000 users / 20,000 sessions) | Own-session list, per-request token lookup, admin active-session list (no sort step), admin audit list (no sort step), audit filter by event, and session purge all use indexes |
| Hot-path SQL (16 checks) | One-statement token validation with idle bump capped at absolute expiry; idle and absolute expiry; unknown and revoked tokens rejected; a locked user's live session keeps working while a disabled user's does not; 12 parallel OTP guesses allow exactly 3; 12 parallel correct submissions consume the code once; resend cap works for an address with no user row |
| Retention purge (6 checks) | A naive challenge purge **fails** on a real foreign-key edge case (a challenge older than 30 days whose session is not). The guarded purge passes and removes the pair on the next run |
| Database roles (18 checks) | `vault_app` can do normal reads and writes, including bigserial tables, but is refused DROP, TRUNCATE, ALTER, CREATE (in both schemas), DROP SCHEMA, CREATE ROLE, and dropping a CHECK. `vault_backup` can only read. A Data-API-style `anon` role cannot read the users table. A table created by a later migration is usable by the app with no new GRANTs |

| | |
|---|---|
| Startup self-check (5 checks) | Passes only for `vault_app`; refuses `vault_migrator`, `vault_backup`, and `postgres`. My first version wrongly accepted the backup role (Postgres treats a comma list of privileges as "any of"), fixed and retested |
| Fallback sync | Under the three roles: the table-scoped dump as `vault_backup` and restore as `vault_migrator` work on the first and second run (default privileges re-grant after `--clean` recreates tables); the app role works on the fallback but still cannot DROP; CHECKs and the composite FK survive; a sabotaged restore exits non-zero and leaves the previous copy intact. A whole-database or schema-scoped dump **fails** (the backup role cannot read other schemas, and the migrator cannot create the schema), which is why the dump is table-scoped. Without `set -o pipefail`, a failed `pg_dump` reports a false success |
| Hashing | argon2id output is `$argon2id$v=19$m=19456,t=2,p=1$...`; correct password verifies and a wrong one is rejected. About 34 ms on my sandbox. Render's free CPU is slower (**verify**; benchmark in week 1) |

**Not yet tested** (needs the application code or the live accounts): the HTTP layer, email delivery, TOTP, the CI workflows themselves, Render and Supabase behaviour. Testing found and fixed six design gaps: a revoke constraint that could not express system revokes, missing indexes for the admin console and purges, a purge that would break on a foreign key, a sync job that could report a false success, a restore that needed a broader grant than least privilege allows, and a privilege self-check that was too permissive.

# 18. Deployment, operations, and disaster recovery

## 18.1 Deployment

- **Backend:** Docker image, deployable to Render free (build from the Dockerfile) or any container host. Configured by env vars only.
- **Database setup, once per database:** run `roles.sql` as the owner on Supabase **and** on Neon, then set the three role passwords out of band.
- **Migrations:** Alembic as `vault_migrator` over `DATABASE_MIGRATOR_URL`, run as a deploy step, not at app start.
- **Frontend:** Vercel (teammate) with the security headers from section 13.
- **CI/CD:** on every push, lint, pytest, build. On main, deploy.

## 18.2 Scheduled jobs (GitHub Actions)

| Job | Frequency (proposed) | Does |
|---|---|---|
| Keepalive | Every 10 min | `GET /health` keeps Render awake and Supabase active. Cron can be delayed, so cold starts remain possible (**verify**) |
| Fallback sync | Every 6 h | Table-scoped `pg_dump` as `vault_backup` piped straight into a single-transaction restore on Neon as `vault_migrator`. **No dump file is stored** (P20, P27). Data-loss window (RPO) up to 6 h |
| Purge | Daily | Runs as `vault_app`. Deletes audit, sessions, challenges, rate-limit and email-send rows older than 30 days, in small batches |

GitHub may disable scheduled workflows on inactive repos (**verify**). The commit history keeps it alive.

## 18.3 Failover runbook (manual)

1. Confirm the primary is down and check the last successful dump time (that is your data loss).
2. Set `DATABASE_URL` to the **Neon `vault_app` URL** (keep it in your password manager) and redeploy. The startup self-check confirms the Neon role is least-privilege.
3. **Revoke all sessions** (`reason = SYSTEM`). A session revoked in the last hours may otherwise come back to life.
4. Lock counters, recent audit rows, and recent registrations after the last dump are lost. Announce it.
5. After recovery, restore forward to the primary and repeat step 3.

### 18.4 Observability

Structured JSON logs with a request ID on every line and no secrets. `/health` reports DB reachability only. The **audit log is a feature**: login success, login failure, lock, unlock, revoke, promote, OTP failure, email sent or failed. The email budget alert fires when 80% of `EMAIL_DAILY_BUDGET` is used.

### 18.5 Disaster recovery

Identities live only in our Postgres, so the dump contains users, hashes, and sessions together. There is no external identity provider to rebuild. Failover accepts that recent data may be missing and forces everyone to log in again.

## 19. Milestones

| Week | Backend + DevSecOps (Shaurya) | Frontend (Apoorva) |
|---|---|---|
| 1 | Repo, CI, Alembic, schema, register, verify email, login, argon2 benchmark, OpenAPI spec shared | Repo, Bootstrap layout, register and login screens against the spec |

| | | |
|---|---|---|
| 2 | Email OTP, sessions, logout, lock, error contract, rate limits | OTP screen, session list, error display |
| 3 | Revocation, admin endpoints, audit, purge, keepalive, backup/ sync jobs, deploy | Admin console, responsive polish. Then (cut first) Expo ID card |
| 4 (buffer) | Failover drill, security testing, README, optional TOTP | Screenshots at phone and laptop width, demo video |

# 20. Evaluation, risks, deliverables

**Success metric (D13).** (1) A revoked session is rejected on its very next request, verified end to end in under 30 s. (2) The CI critical-path suite passes.

**Expected output.** Working demo for End Users and Admins on the deployed backend, README, OpenAPI spec, CI-green repo, and a demo video. Expo app only if time allows.

| Risk | Likelihood | Mitigation |
|---|---|---|
| Render free server sleeps; slow first request | High | Keepalive job; warm up before demos |
| Supabase free project pauses or has no automatic backups (**verify**) | Medium | Keepalive; our own dump job and Neon fallback |
| Resend free cap (100/ day, **verify**) exhausted by abuse | Medium | Per-account throttle, daily budget, alert, TOTP as a no-email path |
| Domain verification takes up to 72 h (**verify**) | Medium | Start in week 1 |

| | | |
|---|---|---|
| Direct DB URL is IPv6-only on free tier, which breaks CI dumps (**verify**) | Medium | Use the session pooler URL for `pg_dump` if needed |
| argon2 memory vs. 512 MB RAM | Medium | Concurrency cap, benchmark in week 1 |
| Supabase Data API exposes our tables (password hashes) through the public anon key | Medium | Private schema `vaultauth`, Data API disabled, no grants to `anon` or `authenticated` (P25, tested) |
| App credential leaks or an injection bug runs hostile SQL | Low | `vault_app` cannot DROP, ALTER, TRUNCATE, or CREATE (tested). Startup self-check enforces it |
| Too much scope for 3 weeks | Medium | Cut list in section 6 |
| Mentor rejects deviations | Medium | Section 2 lists each with its reason |
| Graders need a real inbox for OTP | Low | Graders self-register with their own email (D16). Both admins (Shaurya, Mahati Ma'am) have real inboxes |

**Deliverables.** Repo with clean commits; deployed API and frontend links; README with phone and laptop screenshots; this PRD; OpenAPI spec; CLI bootstrap and reset instructions; demo video.

# 21. Decision record

## 21.1 Decided (approved)

| ID | Decision | Trade-off accepted |
|---|---|---|
| D1 | Static frontend + FastAPI backend + Postgres as source of truth | About a week more than localStorage-only |
| D2 | Passwords hashed server-side with **argon2id only** | Memory-hungry on a 512 MB host |
| D3 | Custom auth, no Supabase Auth | More custom security code, which stays small and tested |
| D4 | Supabase primary, Neon fallback via **scheduled dump/restore from CI**; manual failover that revokes all sessions | Data-loss window equals the schedule (6 h) |
| D5 | Render free web service, plus a Dockerfile; CI keepalive; no security state in memory | Possible cold start of about a minute |
| D6 | Opaque Bearer tokens (hash-only in DB), 30 min idle / 8 h absolute | Frontend holds the token in JS, so XSS matters |
| D7 | Public registration creates `user` only; admins seeded or promoted | Needs a bootstrap path (open) |
| D8 | 3 failures → `LOCKED` 5 min, auto-unlock and admin unlock; one generic login error | Weaker UX, stronger privacy |
| D9 | Email OTP default; optional TOTP replaces it at login | Users without email access need an admin |
| D10 | `PENDING` until emailed code verified; generic "check your email"; resend cap 3 then cooldown; per-IP rate limit | One email per registration against the daily cap |
| D11 | Resend behind an `EmailSender` interface, on a verified domain | Domain DNS verification, 100/day cap (**verify**) |

| | | |
|---|---|---|
| D12 | 30-day purge of audit logs and expired sessions | Older security history is lost |
| D13 | Success metric: revocation on next request (end to end under 30 s) + CI suite passes | |
| D14 | Cut order: Expo → TOTP → Neon sync | |
| D15 | Seed, admin promotion, and reset are **CLI only**, never over HTTP | Graders cannot click a reset button |
| D16 | Graders self-register with their own email. **Shaurya and Mahati Ma'am are the two admins** | Admin codes go to those two real inboxes |
| D17 | API contract = OpenAPI spec generated in CI, plus request/response examples in `auth.md` | `/docs` is off in production, so Apoorva downloads the spec from CI |
| D18 | Three database roles: `vault_app` (DML only), `vault_migrator` (owns schema, DDL, TRUNCATE), `vault_backup` (SELECT only) | One-time setup; the Supabase pooler login format may change with the role name (**verify**) |
| D19 | Email is sent **inside the request**; a send failure returns `503 ERR_EMAIL_UNAVAILABLE` | Provider latency added to register and login; no silently lost codes |
| D20 | Frontend hosted on **Vercel** with real security headers | Extra account and deploy config, in exchange for a real CSP |

## 21.2 Proposed (pending confirmation)

| ID | Proposal |
|---|---|
| P1 | Password policy: 12 to 128 bytes, no composition rules |

| | |
|---|---|
| P2 | OTP: 6 digits, 10 min, hashed, single-use. Resend cooldown 15 min after 3 sends |
| P3 | Tokens: 256-bit random; SHA-256 of the token stored |
| P4 | TOTP secrets encrypted at rest with `TOTP_ENC_KEY`; recovery codes stored hashed |
| P5 | Composite FKs and CHECKs as in section 10; lock order for any multi-table write is `users` → `otp_challenges` → `sessions`; the purge deletes children only, in small batches, so it cannot deadlock with API writes |
| P6 | Migrations use `DATABASE_MIGRATOR_URL` and `pg_dump` uses `DATABASE_BACKUP_URL`, both direct or session; the app uses the pooled transaction-mode URL; no advisory locks or LISTEN/NOTIFY in the app |
| P7 | Open vs. authenticated endpoint list in 11.1; `/docs` off in production; CORS allowlist from env |
| P8 | pytest against real Postgres in CI |
| P9 | Structured JSON logs with request IDs |
| P10 | Non-admin touching another user's session gets `404`, not `403`, so session IDs cannot be probed |
| P11 | Dummy argon2 verification for unknown emails to equalize timing; register always hashes |
| P12 | Correct password on a `PENDING` account returns `VERIFY_EMAIL_REQUIRED` |
| P13 | Login failures do not count OTP failures toward the account lock (OTP has its own 3-try limit and send throttle) |
| P14 | **Admin bootstrap:** both admins register through the normal flow, verify their email, then are promoted with `promote-admin --email`. The CLI never handles a password, so Shaurya never knows Mahati Ma'am's. (No password-reset flow exists, so seeding her account would leave a password Shaurya chose) |

| | |
|---|---|
| P15 | Audit log stores the attempted email for failed logins (lowercased, truncated to 254 chars, escaped on display), purged at 30 days |
| P16 | Resend and register caps are keyed by a SHA-256 **hash of the email** in `rate_limits`, so an unknown address behaves exactly like a real one |
| P17 | `reset` CLI truncates all application tables (not the migration table). The operator must type the database host name to confirm. After a reset both admins re-register and are re-promoted |
| P18 | Session validation is one SQL statement that also bumps idle expiry (capped at absolute expiry). Lock does not invalidate sessions; `DISABLED` does |
| P19 | Admins cannot change their own role or status |
| P20 | The fallback sync pipes `pg_dump` into `psql --single-transaction` and **stores no dump file**. A dump holds emails and password hashes, so it must not become a downloadable CI artifact. A failed restore leaves the previous fallback copy intact (tested). The step must run with `set -o pipefail`, otherwise a failed `pg_dump` reports a false success (tested) |
| P21 | When argon2 parameters in env are raised, existing hashes are re-hashed on the user's next successful login (same algorithm, no downgrade) |
| P22 | One uvicorn worker (`WEB_CONCURRENCY=1`). argon2 runs in a threadpool behind a semaphore of `HASH_CONCURRENCY`, so a login burst cannot freeze `/health` or the keepalive |
| P23 | Choose the Render and Supabase regions together (same or nearest). Every authenticated request is a database round trip with a write |
| P24 | Capacity note: the keepalive uses about 744 of Render's 750 free hours, so only one always-on Render service fits |

| | |
|---|---|
| P25 | Private schema `vaultauth` owned by `vault_migrator`; nothing of ours in `public`; Supabase Data API disabled; no grants to `anon` or `authenticated`. Tested with a Data-API-style role. (Row Level Security as a further backstop is not included. It needs a policy per table and per role, which adds risk for little gain once the schema is private) |
| P26 | The app uses the Supabase transaction-mode pooler with prepared statements off (`prepare_threshold=None`). Transaction mode does not support prepared statements (verified in Supabase's docs) |
| P27 | The fallback sync is table-scoped: `pg_dump -t 'vaultauth.*' --no-owner --no-privileges --clean --if-exists` as `vault_backup`, restored as `vault_migrator`. `roles.sql` must run on Neon first. Tested, and the only variant that works under least privilege |
| P28 | Vercel headers: CSP with `script-src 'self'`, `connect-src` limited to the API origin, `frame-ancestors 'none'`, `base-uri 'none'`, plus `Referrer-Policy`, `X-Content-Type-Options`, `Permissions-Policy`. Bootstrap self-hosted or loaded with SRI |
| P29 | Startup self-check: in production the app refuses to start unless its database role is non-superuser, cannot create objects, cannot TRUNCATE, and has SELECT, INSERT, UPDATE, DELETE on `users`. Tested against four roles |
| P30 | CI secrets are scoped per workflow (GitHub environment secrets): the migrator URL only to the deploy workflow on `main`; the backup and fallback URLs only to the sync workflow; the app URL only to the purge workflow |

## 21.3 Open decisions

| ID | Question | Notes |
|---|---|---|
| O1 | **Sender address and domain** for Resend (for example `no-reply@<your domain>`) | Needed before week 1 DNS work |

| | | |
|---|---|---|
| O2 | Hard cap on total user accounts to protect the email budget? | Not discussed yet. Proposal: no cap, rely on per-IP limits |

*Resolved since the first draft: reset and bootstrap (D15), grader logins (D16), API contract (D17).*

## 21.4 Facts to verify before the final docs

| Topic | What to confirm |
|---|---|
| OWASP argon2id minimum | 19 MiB memory, 2 iterations, 1 parallelism |
| Supabase free | Pause after one week of inactivity; no automatic backups or PITR on free; direct connection is IPv6-only; session pooler works with `pg_dump` |
| Neon free | 0.5 GB storage, 100 compute-unit hours, scale to zero |
| Render free | 512 MB RAM, sleep after 15 min, about 1 min wake, SMTP blocked |
| Supabase roles and exposure | Pooler login format for custom roles (`role.projectref`), that `pg_dump` works through the session pooler, that this project does not auto-expose new tables to the Data API (a 2026 default change), and that the Data API can be switched off |
| Vercel and Render | Vercel config supports the response headers in P28; Render free regions and the 750-hour allowance |
| Resend free | 100 emails/day, 3,000/month, verified domain needed to send to others, 72 h max verification, 1 vs. 3 free domains |
| GitHub Actions | Cron delays; scheduled workflows disabled after 60 days of repo inactivity |

## 22. Deliberately not decided

Real WebAuthn/FIDO2 (reading card only), SMS, account deletion, password reset by email, and password change. Password reset is the likeliest next request, since a locked-out user with a lost password has no recovery path today other than an admin. I recommend listing it as post-demo work.

## 23. References

MDN (fetch, SubtleCrypto for background only), FastAPI, `argon2-cffi`, OWASP Password Storage Cheat Sheet, Bootstrap 5, Render, Supabase, Neon, and Resend documentation. Dataset and research references from v1.0 are unchanged.

## 24. Sign-off

| Role | Name | Date | Approved |
|---|---|---|---|
| Student (backend + DevSecOps) | Shaurya Jain | Week 1 | [ ] |
| Student (frontend) | Apoorva Bhardwaj | Week 1 | [ ] |
| Mentor | Mahati Ma'am | Week 1 | [ ] |